

# Complete Request Flow Architecture

Frontend → Nginx → Starlette + Eureka + APM → Database → Response

## STEP 1 User Action

User interacts with the browser UI

Browser at https://192.168.181.58 triggers GET /py/showAllEnrollEntity

## STEP 2 Frontend App

React/JS app builds the HTTP request

Attaches Token, X-API-Key, Guest, Tune headers. Sets Origin & Content-Type

## STEP 3 CORS Preflight

Browser checks cross-origin permission

OPTIONS request sent first → server responds with Access-Control-Allow-Origin: \*

## STEP 4 Nginx Reverse Proxy

Receives on 192.168.yy.XX:89 → forwards to backend

SSL termination, /py/\* → 192.168.181.170:8999, adds HSTS & security headers

## STEP 5 Starlette Backend

Python app at 192.168.181.170:8999

APM middleware captures timing → validates auth → processes request → queries DB

## STEP 6 Database

Fetches enrolled entity data

Query executed, results serialized to JSON. APM records query duration

## STEP 7 Response Back

JSON travels: DB → Backend → Nginx → Browser

Status 200, chunked transfer, no-cache, CORS headers. APM logs total time

Spring Eureka

SERVICE DISCOVERY

Registers on startup:  
app\_name='Sudice'  
Heartbeat every 30s  
Gateway finds backend IP  
No heartbeat → DOWN

Elastic APM

PERFORMANCE MONITORING

Starlette middleware  
Captures: time, status,  
errors, DB queries  
Ships → Elasticsearch  
Visualize in Kibana

Nginx PROXY

IN the request path

Eureka DISCOVERY

OFF the path — discovery

Elastic APM MONITORING

OBSERVES the path